First-Second transposition sort

+

Scheduling Loops

+

Tree Search

First-Second transposition sort

- The algorithm follows the divide-and-conquer methodology.
- The underlying idea is that we process the given array by iterating two phases.
- We call them the **odd phase** and the **even phase**.

Odd phase: we compare all the odd-indexed elements with their immediate successors in the array and swap them if they're out of order.

Even phase: we repeat this process for all even-indexed elements and their successors.

Then, we iterate over these two phases until we find no more elements to swap.

At that point, we exit the algorithm since the array is sorted.

transpositions refers to wapping/exchanging two adjacent elements when they are out of order.

First-Second transposition sort

Example

Let's say the given array is $arr = [19, 2, 72, 3, 18, 57, 603, 490, 45, 101]$.

Odd phase: Sort all the pairs that start at the odd positions.
Swap 19 and 2, 72 and 3, and 603 and 490. we get

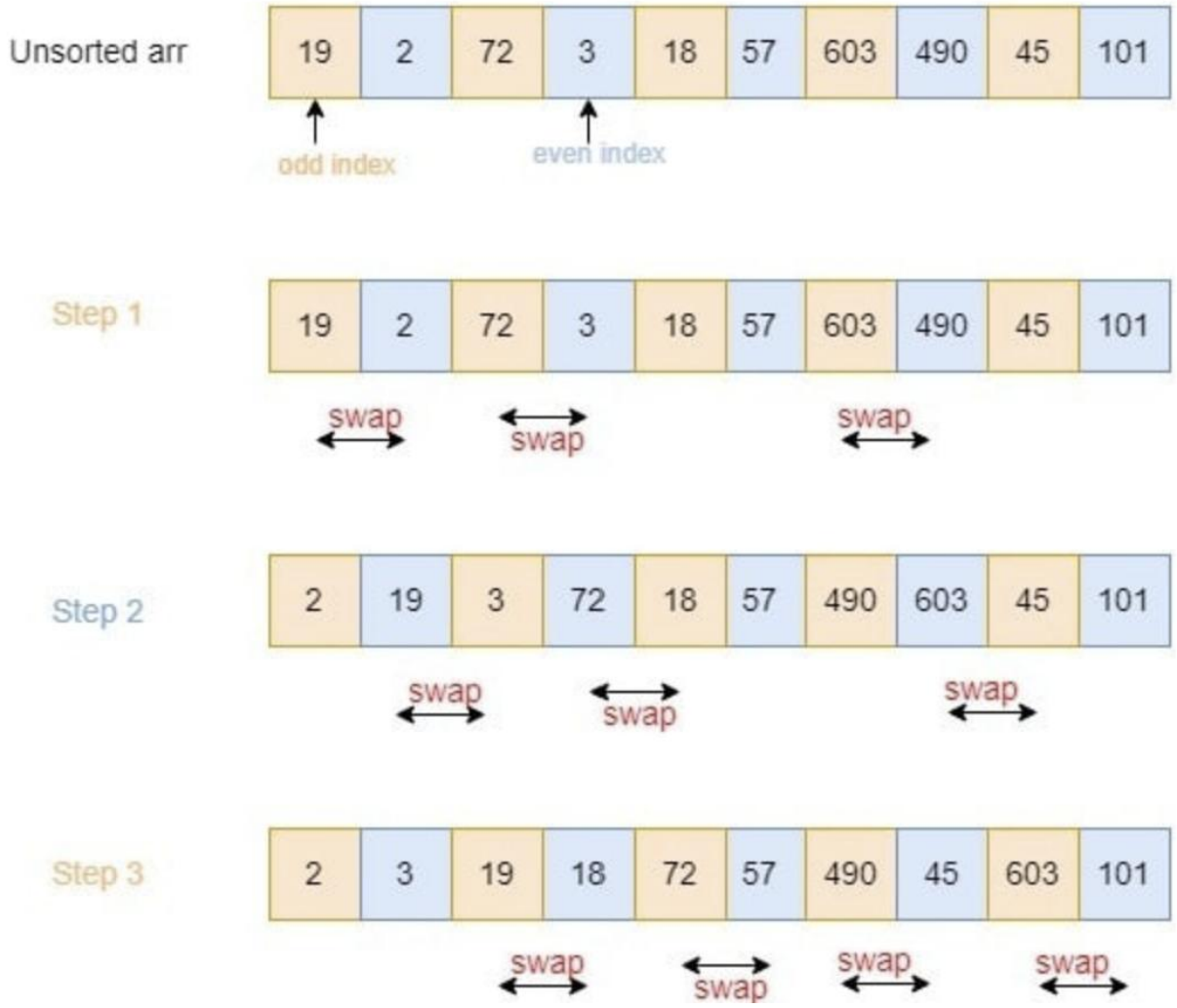
$$arr = [2, 19, 3, 72, 18, 57, 490, 603, 45, 101].$$

Even phase: Sort all the pairs starting at the even positions. In particular, we swap 19 and 3, 72 and 18, and 603 and 45. we have

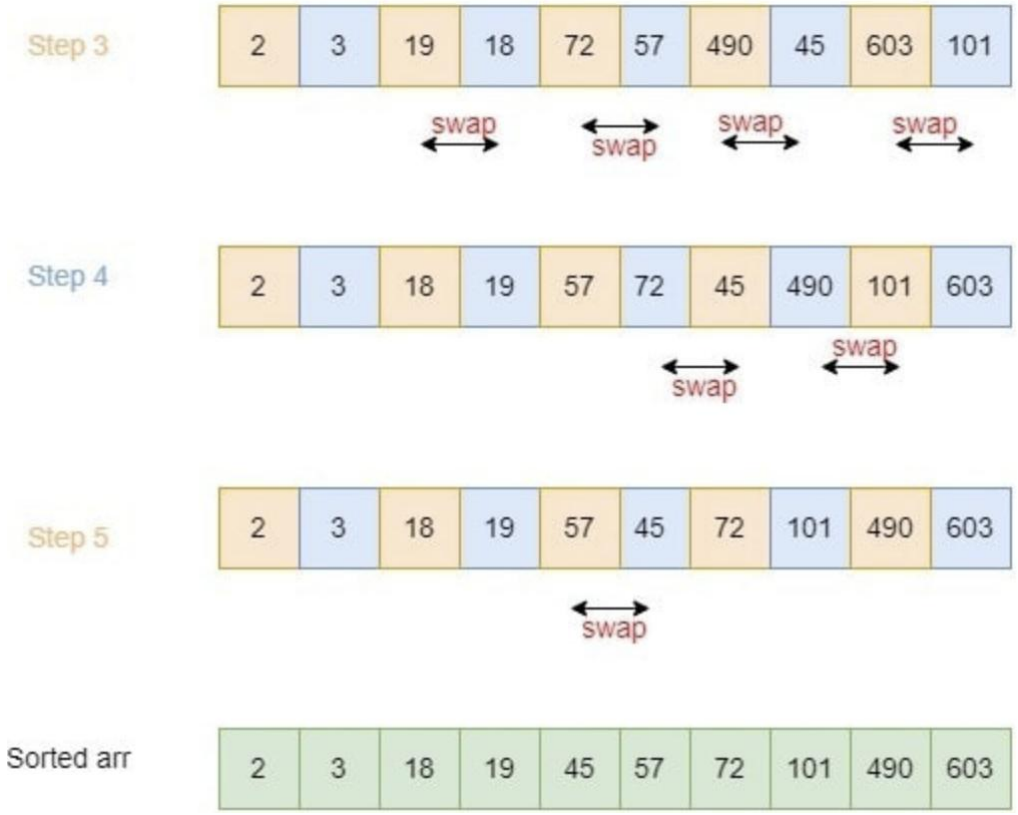
$$arr = [2, 3, 19, 18, 72, 57, 490, 45, 603, 101].$$

We repeat odd and even phases at subsequent steps until we sort the entire array:

First-Second transposition sort



First-Second transposition sort



First-Second transposition sort

```
#include <stdio.h>   #include <omp.h>

int main()
{
    int a[] = {9,7,8,5,6,4,3,1};
    int n = sizeof(a)/sizeof(a[0]);

    omp_set_num_threads(4);

    for(int phase=0; phase<n; phase++)
    {
        if(phase % 2 == 0)
        {
            // First Transposition (Even Phase)
            #pragma omp parallel for
            for(int i=0; i<n-1; i+=2)
            {
                if(a[i] > a[i+1])
                {
                    int temp = a[i];
                    a[i] = a[i+1];
                    a[i+1] = temp;
                }
            }
        }
```

```
        else
        {
            // Second Transposition (Odd Phase)
            #pragma omp parallel for
            for(int i=1; i<n-1; i+=2)
            {
                if(a[i] > a[i+1])
                {
                    int temp = a[i];
                    a[i] = a[i+1];
                    a[i+1] = temp;
                }
            }
        }
    }

    printf("Sorted Array:\n");

    for(int i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}
```

Sorted Array: 1 3 4 5 6 7 8 9

First-Second transposition sort

Thread Assignment Example

First Transposition Phase

Thread	Pair Compared
Thread 0	(1,2)
Thread 1	(3,4)
Thread 2	(5,6)
Thread 3	(7,8)

Second Transposition Phase

Thread	Pair Compared
Thread 0	(2,3)
Thread 1	(4,5)
Thread 2	(6,7)

First-Second transposition sort

Advantages

- Very simple parallel sorting algorithm.
- Easy to implement with OpenMP.
- Uses no extra memory (in-place).
- Stable sorting algorithm.
- Demonstrates the concepts of parallel compare-and-swap and synchronization effectively.

Limitations

- Time complexity remains **$O(n^2)$**, making it inefficient for large datasets.
- Requires synchronization after every phase, which limits scalability.
- Generally slower than parallel Merge Sort, Quick Sort, or Sample Sort for large inputs.

Scheduling Loops in OpenMP

When a loop is parallelized using *#pragma omp for*, OpenMP must decide **which iterations are executed by which thread**. This process is called **loop scheduling**.

Proper scheduling:

- Improves CPU utilization.
- Balances workload among threads.
- Reduces execution time.
- Prevents some threads from becoming idle while others continue working.

Basic Syntax

```
#pragma omp parallel for schedule(type[, chunk_size])
for(int i = 0; i < N; i++)
    {
       // Loop body
    }
```

Parameter	Description
type	Scheduling policy
chunk_size	Number of iterations assigned at one time (optional)

Scheduling Loops in OpenMP

Types of Scheduling

- Static Scheduling
- Dynamic Scheduling
- Guided Scheduling
- Auto Scheduling
- Runtime Scheduling

Static Scheduling

- Iterations are divided **before execution starts**.
- Each thread receives its assigned iterations only once.
- It has **very little scheduling overhead**.

Syntax

```
#pragma omp parallel for schedule(static,2)
```

Scheduling Loops in OpenMP

Static without Chunk Size

```
#include <stdio.h>
#include <omp.h>

int main()
{
   #pragma omp parallel for schedule(static)
   for(int i=0;i<16;i++)
   {
      printf("Thread %d executes iteration %d\n",
            omp_get_thread_num(), i);
   }
}
```

Iterations
0 1 2 3 | 4 5 6 7 | 8 9 10 11 | 12 13 14 15

Thread 0
Thread 1
Thread 2
Thread 3

Static with Chunk Size

```
#pragma omp parallel for schedule(static,2)
```

Thread	Iterations
T0	0–1, 8–9
T1	2–3, 10–11
T2	4–5, 12–13
T3	6–7, 14–15

Scheduling Loops in OpenMP

Dynamic Scheduling

Iterations are assigned **during execution**.

When one thread finishes its assigned chunk, it requests another.

```
#pragma omp parallel for schedule(dynamic, 2)
for(int i=0;i<16;i++)
{
    work(i);
}
```

Initially

T0 -> 0 1

T1 -> 2 3

T2 -> 4 5

T3 -> 6 7

Suppose Thread 1 finishes first.

T1 -> 8 9

If Thread3 finishes next

T3 -> 10 11

Scheduling Loops in OpenMP

Guided Scheduling

Similar to Dynamic Scheduling, but the chunk size decreases over time.

Initially: Large chunks

Later: Small chunks

Syntax

```
#pragma omp parallel for schedule(guided,2)
```

Thread0 -> 0-24

Thread1 -> 25-49

Thread2 -> 50-74

Thread3 -> 75-99

As threads finish, they receive progressively smaller chunks.

20 iterations

↓

10 iterations

↓

5 iterations

↓

2 iterations

↓

1 iteration

Scheduling Loops in OpenMP

Auto Scheduling

The compiler/runtime chooses the scheduling strategy automatically.

Syntax: #pragma omp parallel for schedule(auto)

Runtime Scheduling

The scheduling policy is determined at execution time using the environment variable OMP_SCHEDULE.

Syntax: #pragma omp parallel for schedule(runtime)

```
export OMP_SCHEDULE = "dynamic, 4"
```

Scheduling Loops in OpenMP

Comparison of Scheduling Types

Schedule Type	Work Distribution	Overhead	Load Balancing	Best For
Static	Fixed before execution	Very Low	Poor	Uniform workloads
Dynamic	Assigned on demand	High	Excellent	Irregular workloads
Guided	Large chunks initially, then smaller	Medium	Very Good	Large loops with varying work
Auto	Chosen by compiler/runtime	Depends	Depends	General-purpose use
Runtime	Set using OMP_SCHEDULE	Medium	Depends on selected policy	Performance tuning and experimentation

Scheduling Loops in OpenMP

Choosing the Right Scheduling Policy

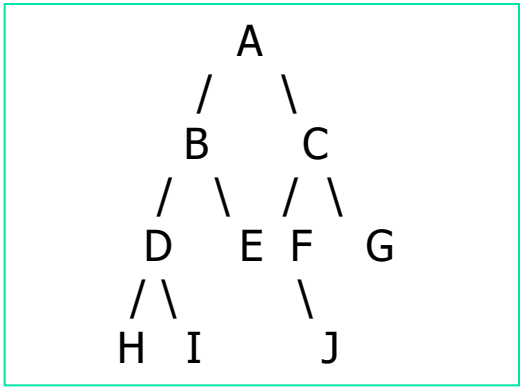
Situation	Recommended
Every iteration performs roughly the same amount of work	static
Some iterations take much longer than others	dynamic
Large loop with gradually changing workload	guided
Want the compiler/runtime to optimize automatically	auto
Want to switch scheduling without recompiling	runtime

Tree Search in OpenMP

A tree search is the process of traversing a tree data structure to locate a node, compute values, or solve a problem. how tree traversal can be parallelized using OpenMP

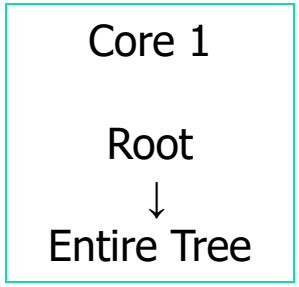
A tree consists of **nodes** connected by **edges**.

- **Root:** A
- **Internal nodes:** B, C, D, F
- **Leaf nodes:** E, G, H, I, J



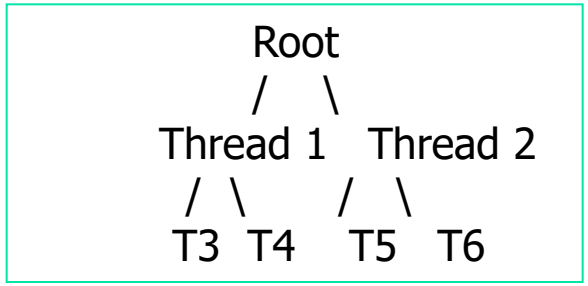
Why Parallelize DFS?
Suppose a binary tree contains **1,000,000 nodes**.

Sequential



Only one processor executes all recursive calls.

Parallel

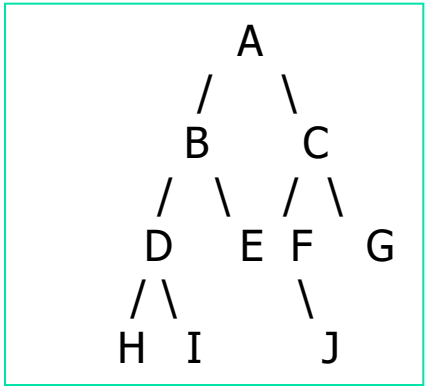


Different branches can be searched simultaneously.

Depth-First Search (DFS)

Depth-First Search (DFS): Searches one branch completely before moving to another.

A → B → D → H → I → E → C → F → J → G



Why OpenMP Tasks?

DFS is recursive, and each subtree is an independent unit of work.

Advantages:

- Dynamic scheduling
- Automatic load balancing
- Efficient recursive parallelism

Task = A unit of work that OpenMP can execute using available threads.

Create a task to search the left subtree.

Create another task to search the right subtree.

Depth-First Search (DFS)

```
#include <stdio.h>          #include <omp.h>

typedef struct Node
{
    int data;       struct Node *left;              struct Node *right;
} Node;

void DFS(Node *root)
{
    if(root == NULL)
        return;
    printf("Thread %d visited %d\n", omp_get_thread_num(), root->data);
    #pragma omp task
    DFS(root->left);
    #pragma omp task
    DFS(root->right);
    #pragma omp taskwait
}

int main()
{
    Node *root = CreateTree();    // User-defined function
    #pragma omp parallel
    {
        #pragma omp single
        {   DFS(root); }
    }

    return 0;
}
```

Breadth-First Search (DFS)

Breadth-First Search (BFS): Visits nodes level by level.

A → B → C → D → E → F → G → H → I → J

Working Principle of BFS

Step 1: Visit the root node.

Step 2: Insert the root into a queue.

Step 3: Repeat until the queue is empty:

- Remove the front node.
- Visit the node.
- Insert all of its unvisited children into the queue.

Breadth-First Search (DFS)

```
#include <stdio.h>                          #include <omp.h>

void BFS(Node *root)
{
   Node *currentLevel[MAX];                    Node *nextLevel[MAX];
   int currentSize = 1;          int nextSize = 0;    currentLevel[0] = root;

   while(currentSize > 0)
   {
      nextSize = 0;

      #pragma omp parallel for
      for(int i = 0; i < currentSize; i++)
      {
         Node *node = currentLevel[i];

         printf("Thread %d visits %d\n",  omp_get_thread_num(), node->data);

         if(node->left)
         {  #pragma omp critical
            nextLevel[nextSize++] = node->left;    }

         if(node->right)
         {  #pragma omp critical
            nextLevel[nextSize++] = node->right;  }
      }

      memcpy(currentLevel, nextLevel,  nextSize * sizeof(Node *));

      currentSize = nextSize;
   }
}
```

BFS vs DFS in OpenMP

Feature	DFS	BFS
Traversal	Depth-wise	Level-wise
Data Structure	Stack / Recursion	Queue
OpenMP Technique	task	parallel for
Parallelism	Task-based	Data parallel
Synchronization	taskwait	critical, atomic, or reductions where appropriate
Best for	Recursive trees	Wide trees and graphs
Scalability	Good	Very good for broad levels